

# Self-Balancing Trees: AVL Rotations

Programming and Data Structures — Week 7, Lecture 3

Dr. Innocent Nyalala

## Objectives

By the end of this lecture, you will be able to state the AVL balance condition precisely and compute any node's balance factor; implement and trace all four AVL rotation cases — left-left, right-right, left-right, and right-left — used to restore balance after an insertion; trace a full sequence of insertions into an AVL tree from empty, including at least one rebalancing rotation; and explain why an AVL tree's height is always  $O(\log n)$ , closing the gap Lecture 2 left open.

## Outline

Today covers a self-correcting bookshelf analogy for active rebalancing; the AVL balance condition, defined precisely in terms of a balance factor; the single rotation, covering the left-left and right-right cases, traced completely; the double rotation, covering left-right and right-left, traced completely; a full sequence of insertions into an initially empty AVL tree, including at least one rebalancing rotation; the proof that AVL height is guaranteed  $O(\log n)$ , resolving Lecture 2's open problem; the MTech-track comparison to red-black trees, a differently-tuned balancing strategy; an in-class quiz; and a summary.

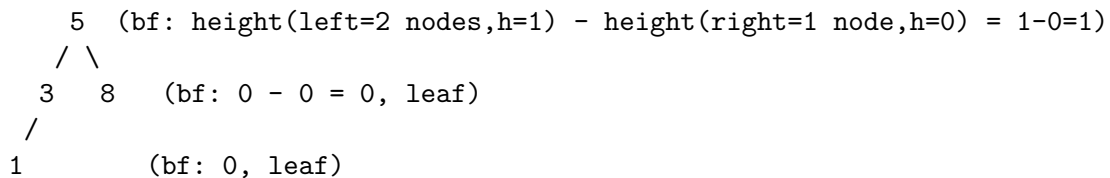
## The self-correcting bookshelf analogy

Picture Lecture 2's plain binary search tree as a bookshelf that never rearranges itself, regardless of how lopsided it becomes as books are added — exactly the degeneration risk that lecture left unresolved. An AVL tree, named for its inventors Adelson-Velsky and Landis, behaves like a bookshelf that automatically checks its own balance and rearranges itself the moment one side becomes too heavy relative to the other. This rebalancing happens immediately after every single insertion or deletion — imbalance is never allowed to accumulate across multiple operations before being corrected.

## The AVL balance condition, precisely

The balance factor of a node is defined as the height of its left subtree minus the height of its right subtree — using the same height definition from Lecture 1. The AVL condition requires every single node's balance factor to be  $-1$ ,  $0$ , or  $+1$ , and nothing more extreme. Whenever an insertion (or deletion) causes some node's balance factor to reach  $-2$  or  $+2$ , a rotation — a local restructuring of the tree — is applied immediately at that node to restore the condition before the operation is considered complete.

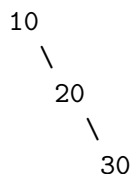
## Computing balance factors on a small tree



Computing balance factors on this three-level tree: the leaves, 1 and 8, each have height 0 and no children, giving a balance factor of 0. Node 3 has a left child (1, height 0) and no right child (conventionally height  $-1$ ), giving a balance factor of  $0 - (-1) = 1$  — within the allowed range. The root, 5, has a left subtree of height 1 (rooted at 3) and a right subtree of height 0 (just the leaf 8), giving a balance factor of  $1 - 0 = 1$  — also within range. This entire tree satisfies the AVL condition.

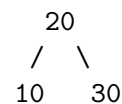
## The single rotation: right-right case

Before (bf of 10 is  $-2$ , unbalanced):



$\Rightarrow$

After (right rotation... actually a LEFT rotation, since heavy on the right):



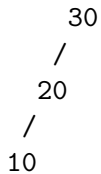
```
def rotate_left(node):
    new_root = node.right
    node.right = new_root.left
    new_root.left = node
    return new_root      # new_root replaces 'node' in the parent's reference
```

The right-right case arises when a node's balance factor reaches  $-2$  (right-heavy) and its right child is itself right-heavy or balanced. A single left rotation fixes this: the right child, 20, becomes the new root

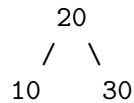
of this subtree, the old root, 10, becomes 20's left child, and any subtree that was previously 20's left child (empty here) is reattached as 10's new right child. This is a constant-time,  $O(1)$  local restructuring — it touches only a small, fixed number of references, regardless of how large the surrounding tree is.

### The single rotation: left-left case, mirrored

Before (bf of 30 is +2):



After (a RIGHT rotation, heavy on the left):

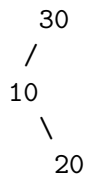


```
def rotate_right(node):
    new_root = node.left
    node.left = new_root.right
    new_root.right = node
    return new_root
```

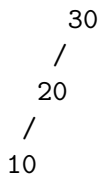
The left-left case is the exact mirror image: a balance factor of +2 (left-heavy) with a left child that is itself left-heavy or balanced. A single right rotation resolves it symmetrically — the left child becomes the new subtree root, and the old root becomes its right child. Comparing `rotate_left` and `rotate_right` directly shows they are structurally identical operations, mirrored — a pattern worth recognizing rather than memorizing as two unrelated cases.

### The double rotation: left-right case

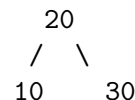
Before (bf of 30 is +2, but LEFT child 10 is right-heavy):



Step 1: rotate\_left on 10's subtree:



Step 2: rotate\_right on 30's subtree:

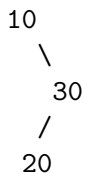


==>

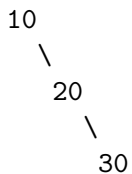
The left-right case is more subtle: the root, 30, is left-heavy (balance factor  $+2$ ), but its left child, 10, is itself right-heavy — a single right rotation on 30 alone would not fix the imbalance, since it does not address the internal skew within the left subtree first. The correct fix performs two rotations in sequence: first a left rotation on the left child's own subtree (rooted at 10), which converts this into the left-left case; then a right rotation on the original node (30), which resolves it exactly as the previous slide's left-left case did. This two-step composition is why it is termed a “double” rotation, though it is simply two of the already-defined single rotations applied in sequence.

### The double rotation: right-left case, mirrored

Before (bf of 10 is  $-2$ , but RIGHT child 30 is left-heavy):

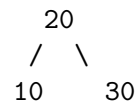


Step 1: rotate\_right on 30's subtree:



==>

Step 2: rotate\_left on 10's subtree:



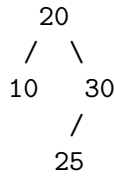
The right-left case mirrors left-right exactly: the root is right-heavy, but its right child is itself left-heavy. The fix again composes two single rotations: first a right rotation on the right child's subtree, converting this into the right-right case, then a left rotation on the original node, resolving it exactly as the right-right case does. All four cases — left-left, right-right, left-right, right-left — reduce to combinations of just the two single-rotation functions already defined.

### A full insertion sequence, traced completely

```

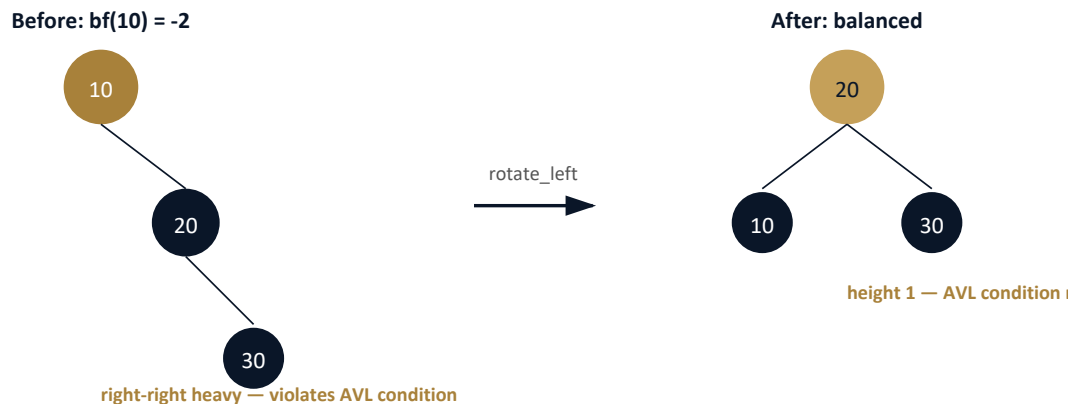
insert 10:  10
insert 20:  10 → right: 20      (bf of 10: -1, OK)
insert 30:  10 → right: 20 → right: 30  (bf of 10: -2!  right-right case)
           rotate_left(10) → new root 20:
               20
              / \
             10  30
insert 25:  20 → right: 30 → left: 25    (bf of 30: +1, OK. bf of 20: still -1, OK)

```



Tracing a complete insertion sequence from an empty tree: inserting 10, then 20, then 30 produces a straight chain, exactly Lecture 1's degenerate case beginning to form — the moment 30 is inserted, node 10's balance factor reaches  $-2$ , triggering the right-right case traced earlier, and a single left rotation immediately restores balance, producing 20 as the new root with 10 and 30 as its children. Inserting 25 next attaches it as 30's left child, and checking balance factors up the tree confirms no further rotation is needed — the tree remains within the AVL condition throughout.

### The picture: before and after the critical rotation



This image captures the single most important moment in AVL tree maintenance: the instant a rotation converts a degenerating chain back into a balanced shape, restoring the height guarantee this lecture proves formally next.

### Why AVL height is guaranteed $O(\log n)$

The proof considers  $N(h)$ , the *minimum* number of nodes an AVL tree of height  $h$  can possibly have — the “thinnest” AVL-valid tree of that height. Since the AVL condition allows a balance factor of up to 1, the thinnest tree of height  $h$  has one subtree of height  $h - 1$  and the other of height  $h - 2$  (the most extreme imbalance still permitted), giving the recurrence  $N(h) = 1 + N(h - 1) + N(h - 2)$ . This recurrence has the same shape as the Fibonacci sequence, which grows exponentially in  $h$ ; solving it (a technique covered formally in Week 9's recurrence analysis) shows  $N(h)$  grows exponentially in  $h$ , which inverts to give  $h = O(\log n)$  for a tree with  $n$  nodes — and critically, this is a *guaranteed*

bound, holding for every possible AVL tree, not merely an average-case or typical-case result the way Week 5's simple sorts' best-case behavior was.

### **MTech addition — red-black trees, a different trade-off**

Red-black trees are a different, widely-used self-balancing strategy, using a looser balance condition based on coloring each node red or black according to specific rules, rather than AVL's strict numeric balance factor. Red-black trees permit slightly taller trees than AVL on average, but require fewer rotations per insertion or deletion to maintain their looser condition. This produces a genuine trade-off: AVL trees favor read-heavy workloads, since their stricter balance gives marginally faster lookups; red-black trees favor write-heavy workloads, since their looser condition means less rebalancing work per update. This is exactly why many languages' standard library ordered map and set implementations (C++'s `std::map`, Java's `TreeMap`) use red-black trees specifically — the write-heavy trade-off suits general-purpose library use better.

### **Rotation cost, precisely**

It is worth being precise about rotation cost. A single rotation involves exactly three reference reassignments — reattaching the new root, moving the displaced subtree, and reattaching the old root as a child — regardless of how large the subtrees involved happen to be, exactly  $O(1)$ . A double rotation is simply two single rotations in sequence, also  $O(1)$  total. Critically, at most one rotation event (single or double) is ever required to restore balance after a single insertion — the imbalance introduced by adding one node is always locally correctable at the first ancestor where a violation is detected while unwinding the recursive insertion, which is why insertion into an AVL tree remains  $O(\log n)$  overall:  $O(\log n)$  to find the insertion point, plus  $O(1)$  for at most one rotation.

### **Exercise**

As an exercise, insert the sequence `[1, 2, 3, 4, 5, 6, 7]` into an initially empty AVL tree, one value at a time, computing balance factors after each insertion and applying whichever rotation (single or double) is triggered. Confirm that the final tree has height 2, not the height 6 degenerate chain a plain BST (Lecture 2) would produce for this exact already-sorted insertion order — direct, concrete evidence of what AVL's active rebalancing actually buys.

### **Comparing all tree structures covered so far**

Placing the plain BST from Lecture 2 directly against this lecture's AVL tree: both cost  $O(h)$  for every operation, but only the AVL tree guarantees  $h = O(\log n)$  in the worst case, at the modest additional cost of up to one  $O(1)$  rotation per update. The plain BST's worst case remains  $O(n)$ , exactly Lecture

1's degenerate chain, whenever insertion order happens to be adversarial (such as already-sorted data) — a risk entirely eliminated by AVL's active rebalancing, at a small, bounded per-operation cost.

### Quiz — identify the rotation case

Given a node with balance factor  $-2$ , whose right child has balance factor  $+1$ , determine which of this lecture's four rotation cases applies, before checking the answer.

### Quiz — answer

A balance factor of  $-2$  indicates the node is right-heavy. Its right child having a balance factor of  $+1$  indicates that child is itself left-heavy. A right-heavy node whose right child is left-heavy is precisely the right-left case defined earlier in this lecture, requiring a double rotation: first a right rotation on the right child's own subtree, converting the situation into the simpler right-right case, then a left rotation on the original node to complete the rebalancing.

### A second quiz check — left-left case

As a second check, given a node with balance factor  $+2$  whose left child has balance factor  $0$ , determine which rotation case applies and whether it requires a single or double rotation, before checking the answer.

### Second quiz — answer

A balance factor of  $+2$  indicates left-heaviness. The left child having a balance factor of  $0$  means it is not itself right-heavy, which is the condition (balance factor  $\leq 0$ ) for the left-left case defined earlier in this lecture. A single right rotation resolves it directly, with no second rotation required — confirming that only a left-heavy child skewed toward the *right* (a positive-turned-negative pattern) would have required the more involved double rotation instead.

### Summary

The AVL condition requires every node's balance factor to remain within  $\{-1, 0, 1\}$ , actively enforced immediately after every insertion or deletion through rotation. Four rotation cases handle every possible imbalance: left-left and right-right resolve with a single rotation, left-right and right-left require a double rotation composing two single rotations. This guarantees  $h = O(\log n)$  for every possible AVL tree, a proven worst-case bound, not merely a typical one — fully resolving the degeneration risk Lecture 2 left open. Red-black trees offer a related but differently-tuned balance-versus-update-cost trade-off, used by many standard library map and set implementations. Next week moves to hash

tables and heaps, structures that trade away the tree's ordering guarantee in exchange for even faster average-case operation costs in specific circumstances.